

→ JDBC in Java:

→ What is JDBC?

JDBC (Java Database Connectivity) is a Java API used to **connect java application** with databases and perform database operations such as:

- Insert
- Select
- Update
- Delete

Eg;

Java Application



JDBC



JDBC Driver



Database

(MySQL / Oracle / etc...)

→ Why JDBC is used?

Suppose we create a **Student Management System** in java.

We want to Store:

Roll No.

Name

Age

Marks

inside a database.

Java alone cannot directly communicate with every database in the same way.

JDBC provides the standard interface b/w java and the database.

Main Uses :

1. Establish database connection
2. Executes SQL queries
3. Insert data
4. Retrieve data
5. Update data
6. Delete data
7. Process query results
8. Handle database-related exceptions.

→ JDBC Architecture :

Java Application



JDBC API



JDBC Driver



Database

→

JDBC API:

JDBC provides classes and interfaces for database communication.

important interface/class include:

Driver

Connections

Statements

PreparedStatement

CallableStatements

ResultSet

DriverManager

SQLException

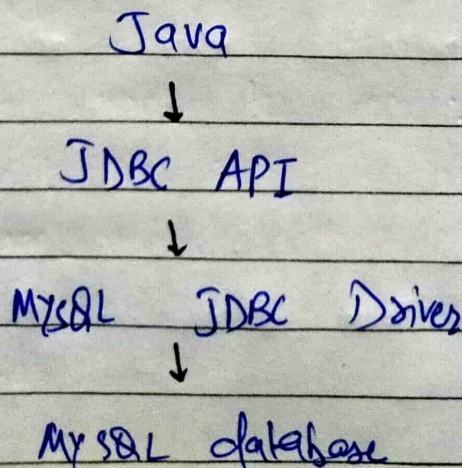
Many JDBC types are available in:

java.sql

JDBC Driver:

A JDBC Driver is software that enables java to communicate with a particular database.

Eg:



→ Steps of JDBC:

The basic JDBC workflow is:

Import JDBC package



Establish connection



Create Statement / Prepared Statement



Execute SQL query



Process ResultSet



Close Resource

Connect → Create → Execute → Process → Close

Step 1: Import JDBC package:

```
import java.sql.*;
```

This provides access to JDBC classes/interface such as:

Connection

Statement

ResultSet

SQLException

etc

Step 2: establish connection:

The ~~Driver~~ **DriverManager** class can be used to establish a connection.

```
Connection con = DriverManager.getConnection(  
    "jdbc:mysql://localhost:3306/college",  
    "root",  
    "password",  
);
```

→ **JDBC URL:**

A JDBC URL identifies the database to which java should connect.

JDBC



JDBC protocol



mysql



Database type



localhost



Database server



3306

MySQL Port



college

Database name

Step 3: Create Statement:

A Statement object is used to execute SQL statements.

```
Statement stmt = con.createStatement();
```

Then ;

```
ResultSet rs = stmt.executeQuery (
    "select * from Student"
);
```

Statement :

Statement is used to execute SQL statement.

Eg ;

```
Statement stmt = con.createStatement();
```

```
ResultSet rs = stmt.executeQuery (
    "Select * from Student"
);
```

→ Step 4: Execute SQL:

JDBC mainly provides three methods for executing SQL.

~~executeQuery~~ executeQuery():

Used for queries that return set, typically SELECT.

```
ResultSet rs = stmt.executeQuery(
    "select * from Student"
);
```

executeUpdate() :

Used for INSERT, UPDATE and DELETE.

Eg :

```
int rows = stmt.executeUpdate(
    "UPDATE Student SET age = 21
    where id = 1"
);
```

execute()

Can execute SQL when the result type may vary.

```
boolean result = stmt.execute(sql);
```

Ques: ResultSet :

ResultSet stores the data returned by a query.

Eg :

```
ResultSet rs = stmt.executeQuery(
    "select * from Student"
);
```

→ Complete JDBC Example:

Suppose MySQL contains:

Database: college

Table: Student

id	name	age
1	Akash	21
2	Ayush	22
3	Aishwarya	20

```
import java.sql.*;
```

```
public class JDBCExample {
```

```
    public static void main (String args[]) {
```

```
        String url =
```

```
        "jdbc:mysql://localhost:3306/college";
```

```
        String username = "root";
```

```
        String password = "password";
```

```
        try {
```

```
            Connection con = DriverManager.getConnection(
                url, username, password
            );
```

```
Statement stmt = con.createStatement();
```

```
ResultSet rs = stmt.executeQuery (
    "select * from Student"
);
```

```
while (rs.next()) {
    cout (
        rs.getInt("id") + " " +
        rs.getString("name") + " " +
        rs.getInt("age")
    );
}
```

```
rs.close();
stmt.close();
con.close();
}
```

```
catch(SQLException e) {
    cout ("Database Error: " + e.getMessage());
}
```

```
}
```

Akash